CSE634-ARP

Discrete Planning

Instructor: Dr. Tanmoy Kundu

August 13, 2026

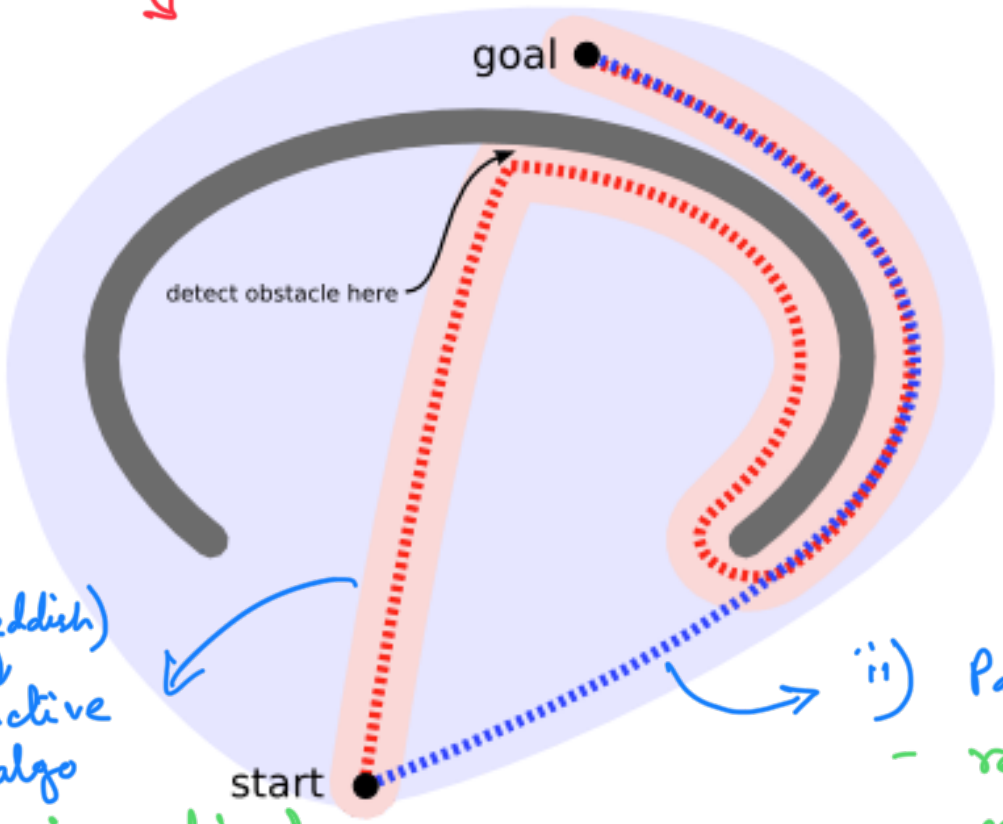
Forward Search

Pathfinding vs Reactive movement

- **Pathfinding:** Plans ahead rather than waiting until the last moment to discover there's a problem
- **Reactive movement:** Movement with some instantaneous realistic events

Example: Situation-1

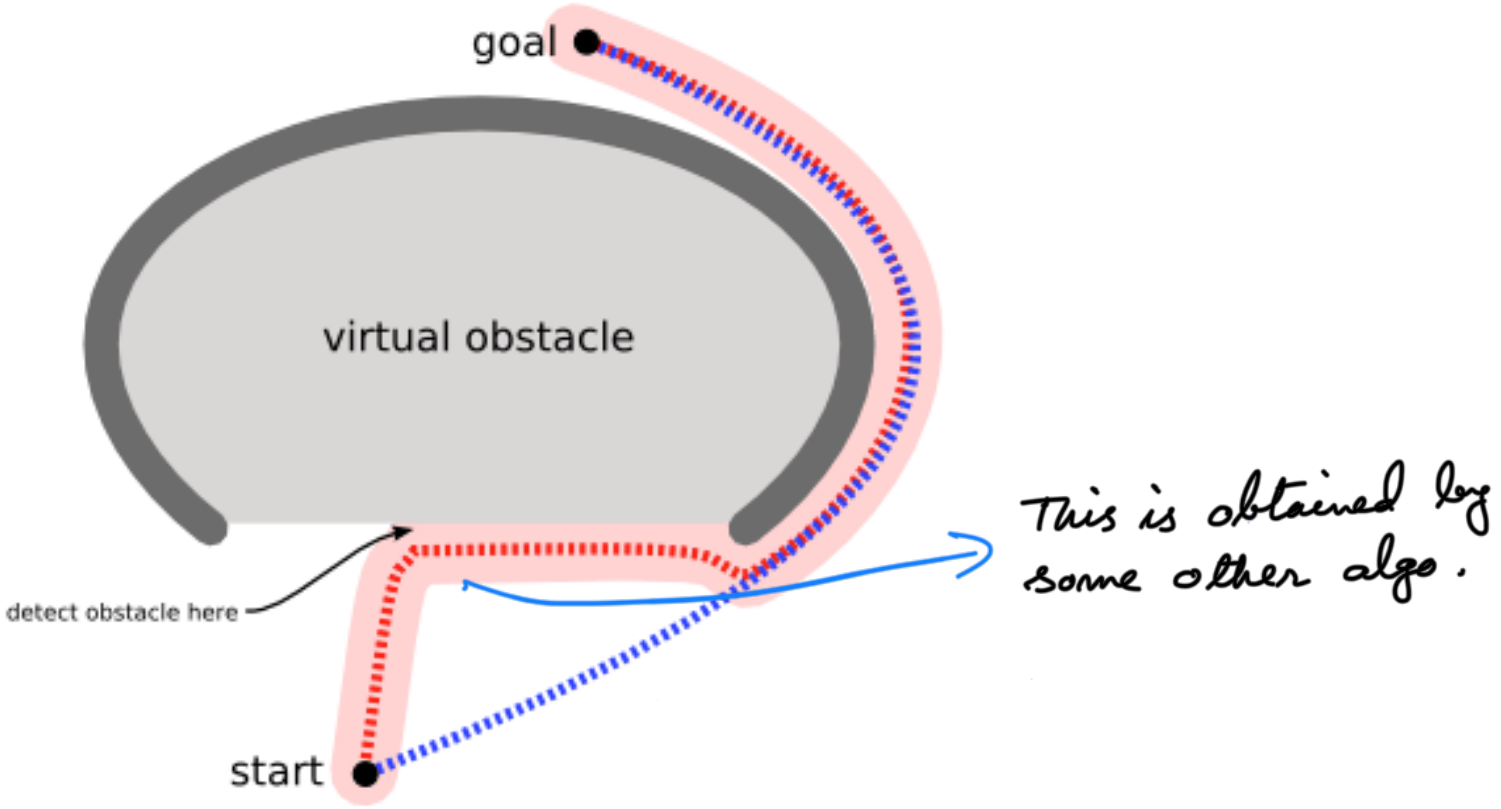
- Two algorithms
  * Performance vs region explored



(in reddish)
i) Reactive algo
- region explored is comparatively low (Pink colored region)
- path is not optimal

(in blueish)
ii) Pathfinding (Planning)
- region explored is much higher
- path is optimal

Example: Situation-2



A tradeoff between planning ahead vs reactive movement

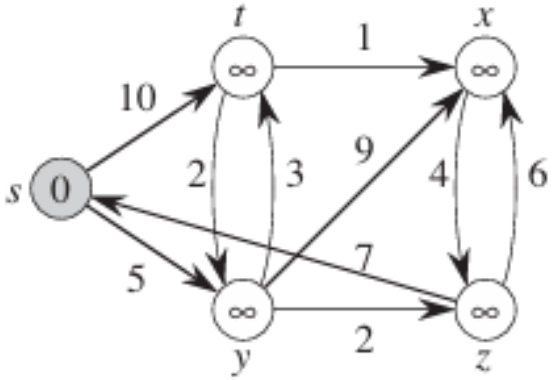
- **Pathfinding:** Plans ahead rather than waiting until the last moment to discover there's a problem
  - It is generally slower but gives better results. (Dijkstra's algorithm)
- **Reactive movement:** Movement with some instantaneous realistic events.
  - Movement is generally faster but can get stuck. If the world is changing often, reactive movement is more valuable. (Best-first algo)
- **Recommended:** *Pathfinding* for big picture, slow changing obstacles, and long paths; and *reactive movement* for local area, fast changing and short paths.

↳ a mixture of the two algorithms can bring together the advantages of these two algorithms [a hint towards the A* algorithm]

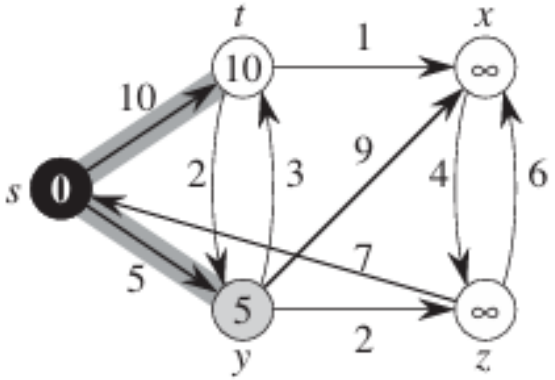
Dijkstra's algorithm

- A greedy algorithm
- Works for a weighted graph with *non-negative* edge weights
- Finds the shortest path from a single source vertex to all other vertices
- **Key idea:**
  - Repeatedly selects the unvisited vertex with the smallest known distance from the source (and not the remaining distance to the goal)
  - Updates the distances of its neighboring vertices
- Guaranteed to find an optimal path

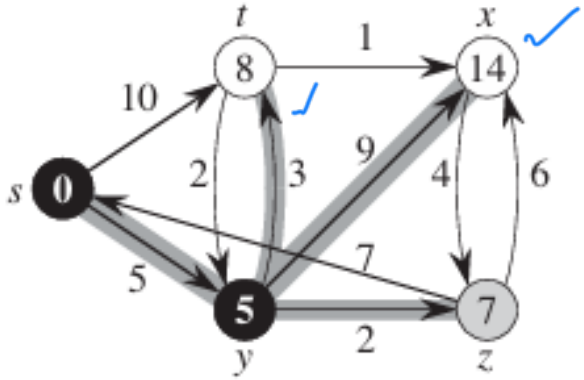
Example: Dijkstra's algorithm



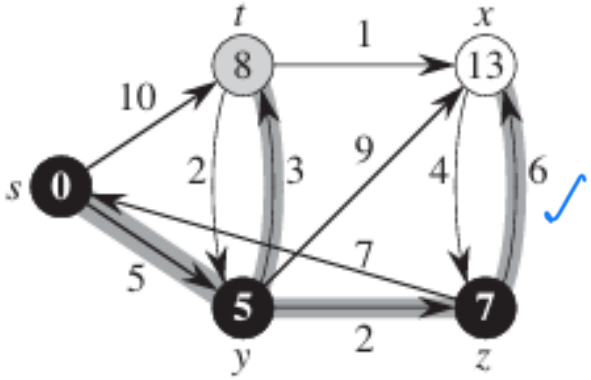
(a)



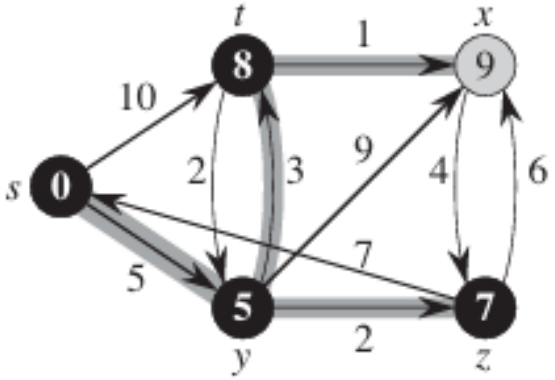
(b)



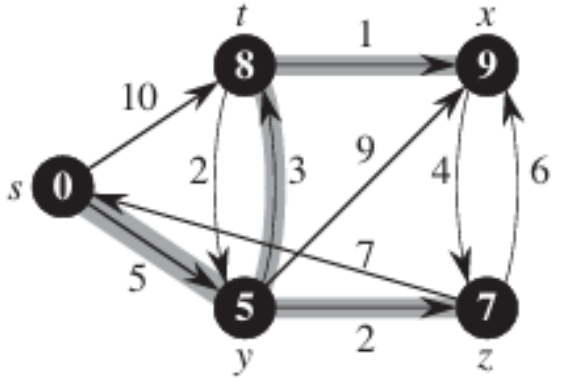
(c)



(d)



(e)

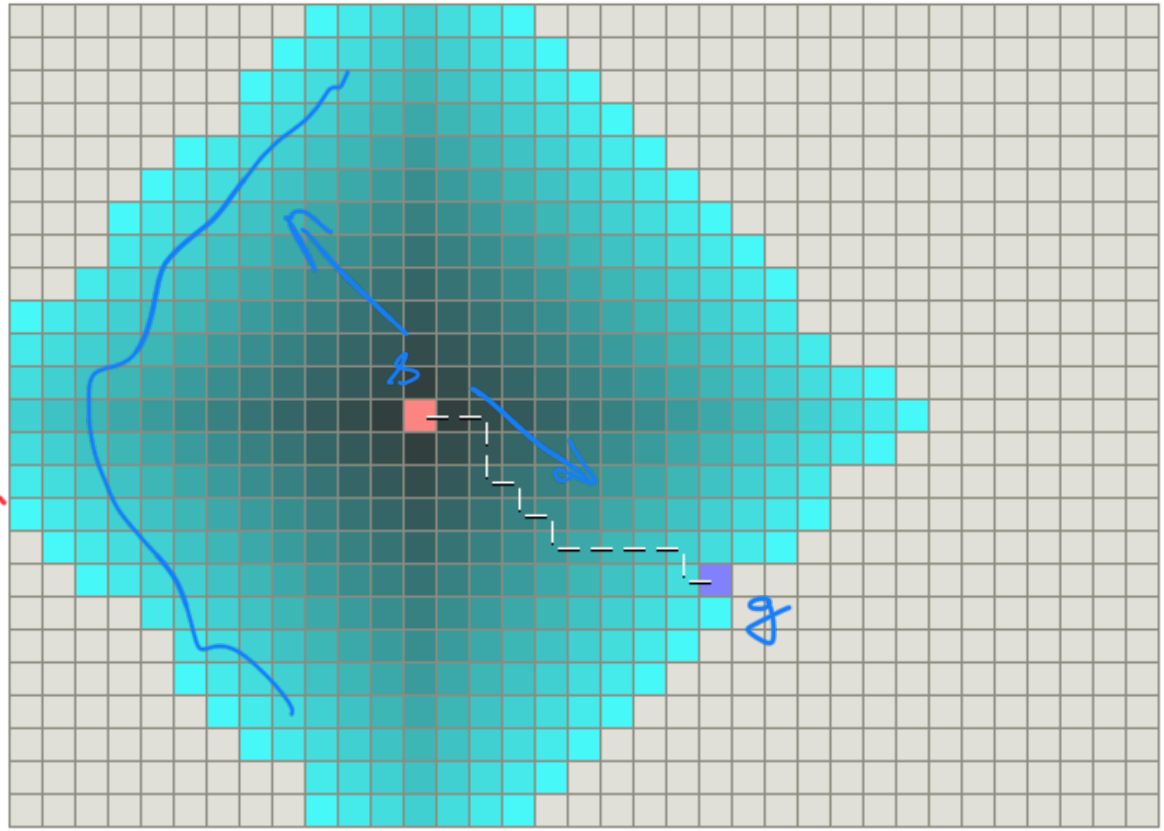


(f)

Mobile robot example for Dijkstra's algorithm: Workspace without obstacles



Motion dynamics



Number of cells traversed by this approach is shown in blueish shade.

Note that, the knowledge of the goal location does not add any advantage here.

Application of Dijkstra's algorithm to find a path from a given source (s) to goal (g). The path is guaranteed to be optimal.

Best-first search

Goal state / goal location is known or informed.

- A heuristic algorithm for graph/tree search
- Always expands the node that currently appears most promising according to an evaluation function [Function f]
- **Evaluation:**
  - $f(n) := h(n)$
  - $h(n)$ **estimates** the remaining cost from node n to the goal
  - The node with the **smallest heuristic value** is expanded first
- **Not guaranteed** to find an optimal path

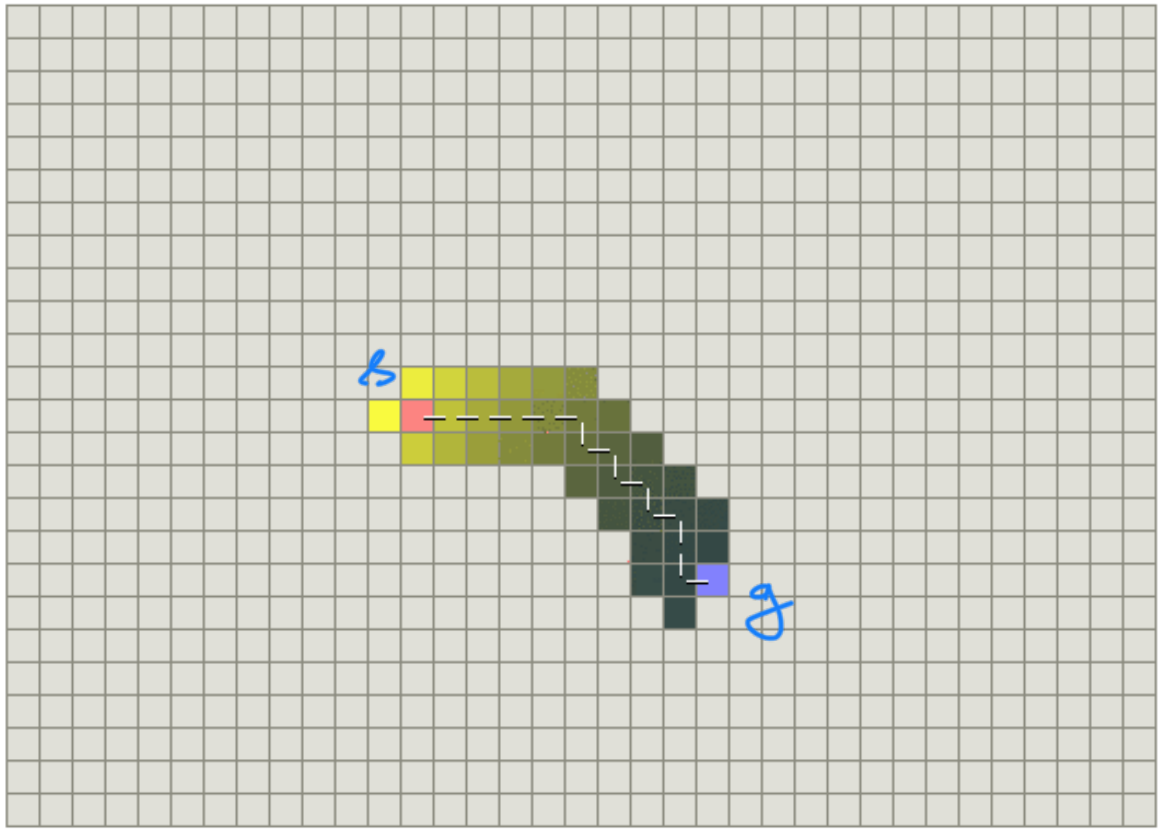


though it does not consider the actual path already traversed from source to node n.

Mobile robot example for Best-first: Workspace without obstacles

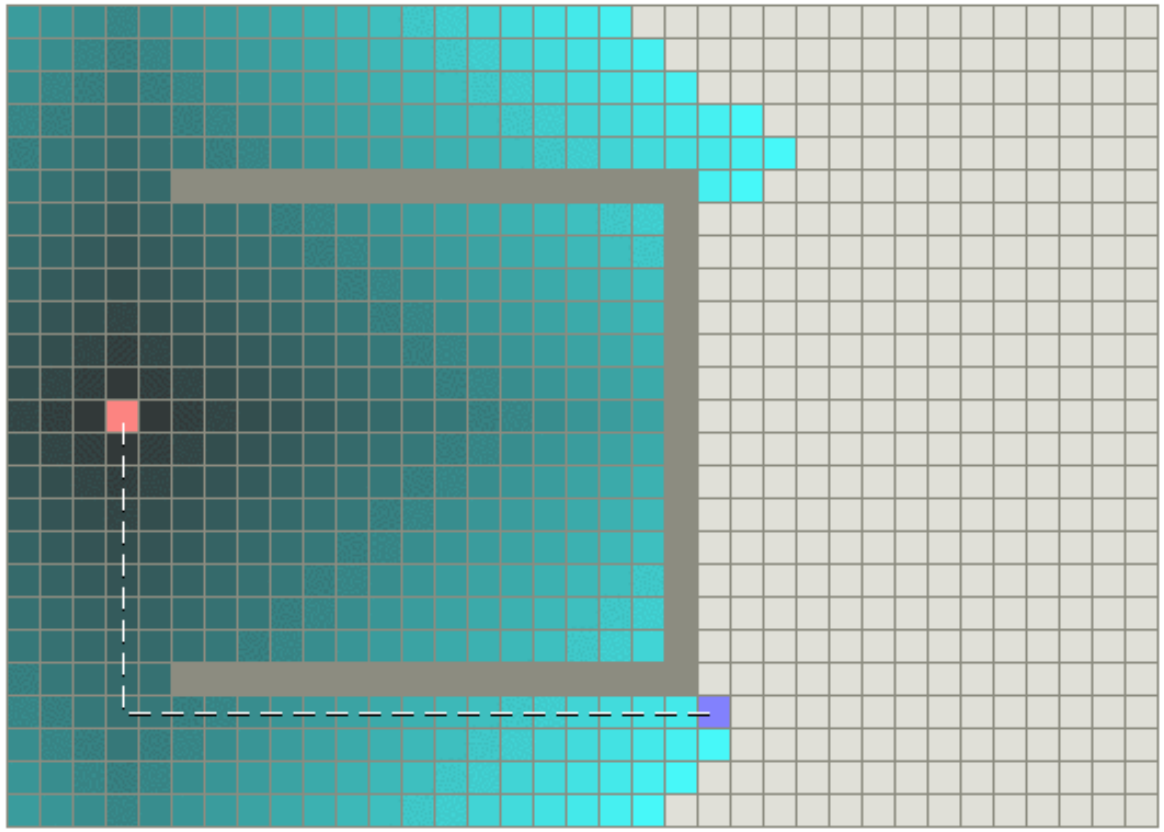
The prior information about the goal state plays a role here.

Number of cells traversed by this approach is significantly lower than the Dijkstra's algo.



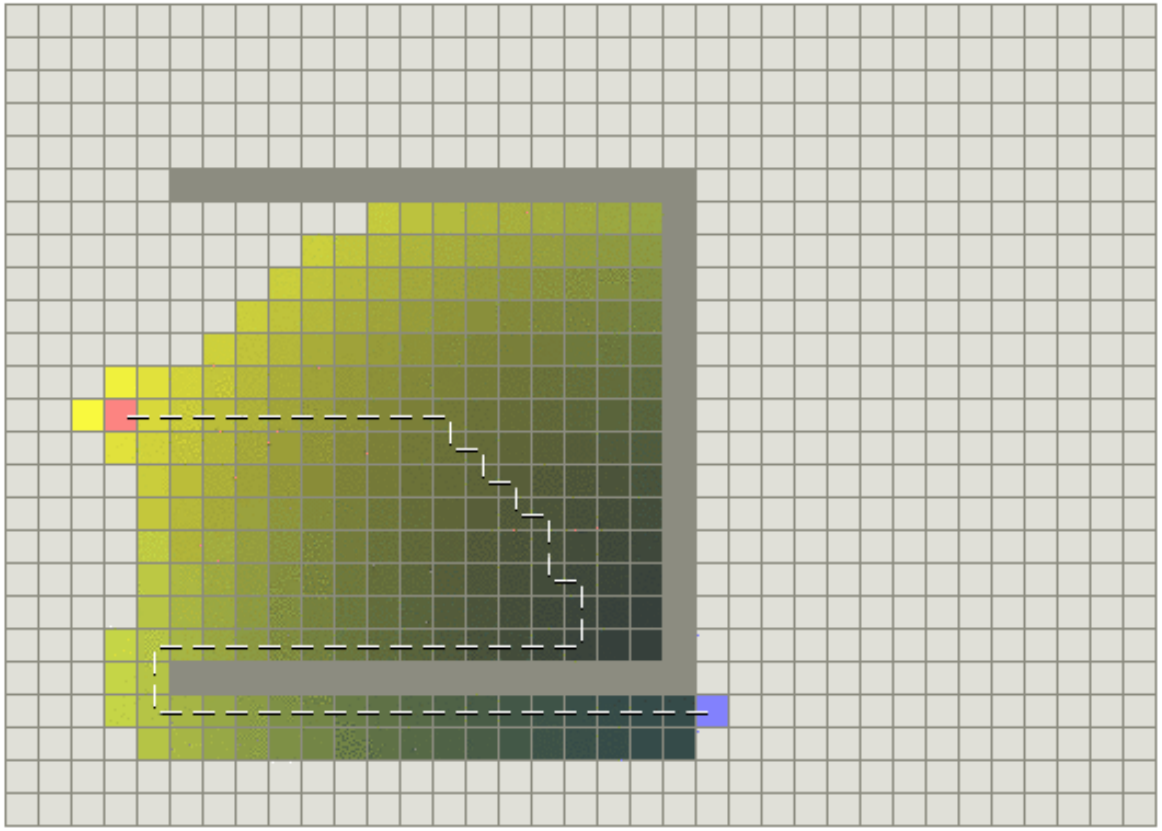
Application of Best-first algorithm for finding a path from s to g. The path is not guaranteed to be optimal.

Example: Dijkstra with obstacles



Dijkstra's algo is applied to another problem (with an addition of obstacles)

Example: Best-first with obstacles



Approach: Best-first search

An example of the heuristic fn.

f: Euclidean distance

f(n): Euclidean dist from n to the goal node.

f represents the estimation here.

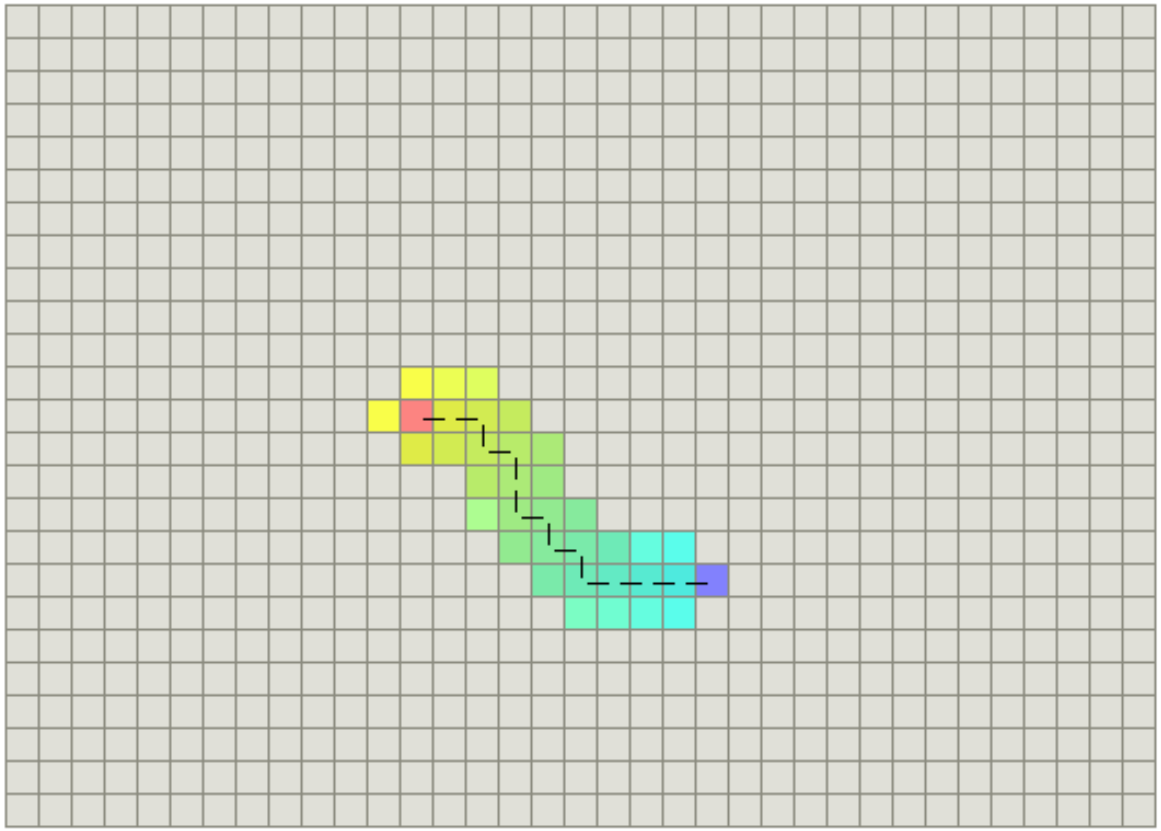
(f could be some other fn as well)

A* search

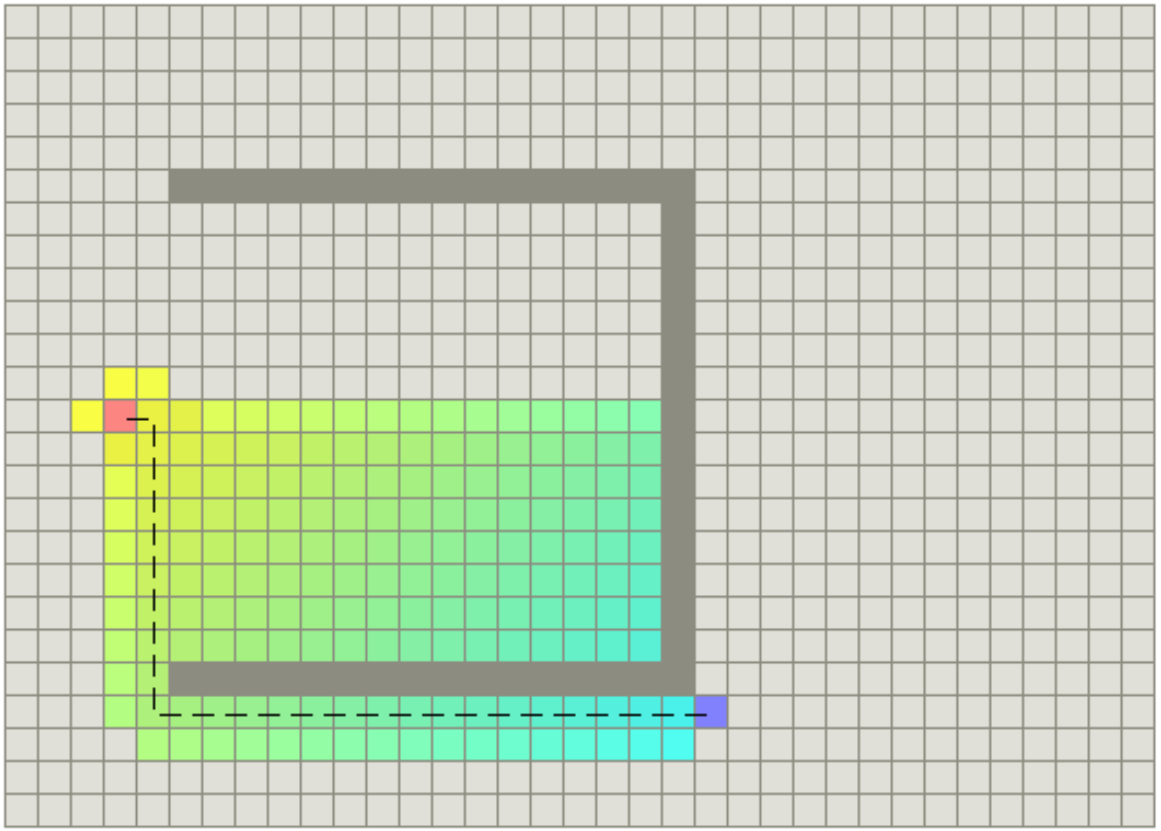
- A heuristic algorithm to find the shortest path between a start and a goal node in a graph
- Combines the concepts of both Dijkstra's algorithm and Best-first search
- Graph traversal algorithm used to find the shortest path between
- **Evaluation:**
  - Evaluates each node using the function $f(n) := g(n) + h(n)$
  - $g(n)$: actual cost from the start node to node n, $h(n)$: estimated cost (heuristic) from node n to the goal, $f(n)$: estimated **total** cost of the path through node n.
  - The node with the **smallest** $f(n)$ is expanded first
- Guaranteed to find the optimal path

Mobile robot example for A*: Workspace without obstacles

The previous problem with a different approach (A* algo)
A* algorithm leverages the previous two algorithms and brings together the advantages of both the algorithms.



Example: A* with obstacles



A general forward search algorithm

```
FORWARD_SEARCH
1   Q.Insert(x_I) and mark x_I as visited
2   while Q not empty do
3       x ← Q.GetFirst()
4       if x ∈ X_G
5           return SUCCESS
6       forall u ∈ U(x)
7           x' ← f(x, u)
8           if x' not visited
9               Mark x' as visited
10              Q.Insert(x')
11          else
12              Resolve duplicate x'
13  return FAILURE
```

X: set of states

x_I: Initial state

$X_G \in X$: set of goal states

$U(x)$: action set at state $x \in X$

f: state transition function

- Uninformed vs Informed search
- Evaluation function f for different algorithms
  - Dijkstra : $f(n) = g(n)$
  - Best first: $f(n) = h(n)$
  - A* : $f(n) = g(n) + h(n)$

f, g and h are as per conventions given in slide 16.

- Time complexity
  - Dijkstra $O\left[(V+E)\log V\right]$
  - Best first $O(b^d)$
  - A* $O(b^d)$

b: branching factor

d: depth of solution

Backward Search

Backward search

- Backward versions of any of the forward search algorithms can be made.
- For many planning problems, the branching factor is large when starting from x_I
  - In this case, it might be more efficient to start the search at a goal state and work backward until the initial state is envountered.

o Backward version of Dijkstra's algorithm.

Backward search

Forward approach

$f: X \times U \rightarrow X$

$f(x, u) = x'$

U: set of possible actions $= \{u_i\}$

$U^{-1} = \{(x, u) \in X \times U \mid x \in X, u \in U\}$

$U^{-1}(x') = \{(x, u) \in U^{-1} \mid x' = f(x, u)\}$

- It may be preferable to pre-compute a representation of transition function f for the "backward" search algorithm.
  - Backward action space
  - Backward action space
  - Perfect symmetry with respect to the forward search

From a given $x' \in X$, the set of all $(x, u) \in U^{-1}$ that map to x' using f.